

Coursework

Autonomous Robot Package Delivery using Unity

Student(s) Name(s):

Santiago Cruz Lopez | **ID:** 2584023

7CS083/UZ2: Intelligent Agents

Online

Professor Adam Worrallo

May 4, 2026

Wolverhampton, UK

The University of Wolverhampton

1. Introduction

This coursework focuses on designing and building a simulation that features an intelligent virtual agent for autonomous robot package delivery. The agent will plan routes through a small city environment, navigate them, and interact with its surroundings in Unity. It will employ the **A*** algorithm combined with reinforcement learning (RL) for route finding from a starting location to a designated node on a waypoint graph. Once the agent reaches its target, it will return to the starting location.

Five interconnected scripts are used in this project to construct a deterministic counterpoint. During **Awake()**, **GridManager.cs** creates a static walkability grid by querying Unity's physics engine via **Physics.CheckSphere** with adjustable layer masks. The area is then visually populated by **GridFiller.cs**, which creates a unified urban cityscape without disrupting the **Pathfinding.cs** logic by placing randomized housing prefabs on unoccupied, non-walkable nodes (Friedman Education, 2023). The process of finding a path. Using an octile-distance heuristic, **Pathfinding.cs** applies **A*** to extract minimum-cost pathways between walkable nodes. **Agent.cs** executes those paths via **Vector3.MoveTowards**, advancing sequentially through each **Node.worldPosition** (Riedl, 2023).

The behavioural loop is coordinated by **DeliveryManager.cs**, which sends the agent to a randomly chosen road-tagged location, logs the amount of time spent on each journey, and then returns the agent to its starting point before starting the subsequent delivery cycle. By contrast, these design decisions create a known-environment agent—full observability, zero uncertainty—exactly the circumstances that make RL's adaptive potential (Riedl, 2023).

2. Reinforcement Learning for Route-Finding

A natural sequential decision problem is route-finding (Sutton & Barto, 2018). In this context, every directional move constitutes an action, and each grid position represents a distinct state (Russell & Norvig, 2022). Rewards play a crucial role in shaping behavior. Although a computationally tractable approach exists, using a sparse reward scheme with a terminal reward given only upon delivery (and zero otherwise) tends to take a long time to converge. In contrast,

denser formulations encourage efficiency by imposing penalties per step that are proportional to the length of the path taken (Sutton & Barto, 2018).

In this project, states correspond to **Node.gridX** and **Node.gridY** coordinates. The action set aligns precisely with the eight neighboring positions returned by **GridManager.GetNeighbors()**, utilizing the walkable boolean stored for each Node to filter out any inadmissible operations.(Juliani, 2020).

The Bellman optimality equation is fundamental in sustaining action-value estimates $Q(s, a)$ in Q-learning, which is Watkins's innovative model-free, off-policy approach (Sutton & Barto, 2018). This equation is expressed as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Under conditions of sufficient exploration and decreasing learning rates, convergence to Q^* is guaranteed. Importantly, a Q-learned policy can adapt implicitly when GridManager.cs is updated to re-query walkability in real-time, transforming the initially static grid into a dynamic environment. This adaptability provides a significant advantage, as opposed to the A^* algorithm, which necessitates comprehensive re-planning with every alteration in the structure (Juliani, 2020).

As state spaces increase in size, tabular Q-learning becomes less effective. Deep Q-Networks solve this issue by utilizing function approximation: a neural network parameterized by weights θ approximates $Q(s, a; \theta)$ (K & V, 2015). This network learns from past experiences through a method called experience replay and uses a fixed target network to make updates more stable (K & V, 2015). It takes input vectors that show if each node is walkable, if it is occupied, and the straight-line distance to the goal. This allows the network to work well with different grid setups for deliveries. These details can be calculated in Node.cs. However, the current Pathfinding.cs has some design issues that make this approach difficult (Sutton & Barto, 2018).

The A^* algorithm finds the shortest path in static and fully observable graphs, ensuring that the path is always optimal. In contrast, reinforcement learning (RL) methods adapt to partly observable environments, have changing costs, and involve uncertainty. The solution in this

project serves as a strong baseline rather than just a placeholder. The `DeliveryManager.cs` records the **timeElapsed** metric as a precise benchmark, allowing us to compare the effectiveness of a trained RL policy to traditional pathfinding methods (Argall et al., 2009).

Unity Assets

- **Free Low Poly Robot:** A character model asset.
- **SimplePoly City P:** An asset pack for creating a city environment.

References

- Argall, B. D., Chernova, S., Veloso, M., & Browning, B. (2009, 05 31). *Robotics and Autonomous Systems*. A survey of robot learning from demonstration.
<https://doi.org/10.1016/j.robot.2008.10.024>
- Juliani, A. (2020, 05 6). *Unity: A General Platform for Intelligent Agents*. Cornell University.
<https://arxiv.org/abs/1809.02627>
- K, K., & V, M. (2015, 02 26). *Human-level control through deep reinforcement learning*. Nature.
<https://doi.org/10.1038/nature14236>
- Riedl, M. (2023). *Grid Navigation and Path Planning*. CS 4731/7632: Game Artificial Intelligence. College of Computing, Georgia Institute of Technology.
<https://faculty.cc.gatech.edu/~surban6/2018sp-gameAI/homeworks/homework1.html>
- Russell, S., & Norvig, P. (2022, 08 22). *Artificial Intelligence: A Modern Approach, 4th US ed.* Berkeley. <https://aima.cs.berkeley.edu/>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
- Friedman Education. (2023). *CS_4731*. Georgia Tech's CS 4731 - Game AI (Summer 2023).
https://github.com/Fried-man-Education/CS_4731